

Do Automated Fixes Truly Mitigate Smart Contract Exploits?

Sofia Bobadilla¹, Monica Jin¹, Martin Monperrus
KTH Royal Institute of Technology
{sofbob,mjin,monperrus}@kth.se

Abstract—Automated Program Repair (APR) for smart contract security promises to automatically mitigate smart contract vulnerabilities responsible for billions in financial losses. However, the true effectiveness of this research in addressing smart contract exploits remains uncharted territory. This paper bridges this critical gap by introducing a novel and systematic experimental framework for evaluating exploit mitigation of program repair tools for smart contracts.

We qualitatively and quantitatively analyze 20 state-of-the-art APR tools using a dataset of 143 vulnerable smart contracts, for which we manually craft 91 executable exploits. We are the very first to define and measure the essential “exploit mitigation rate”, giving researchers and practitioners a real sense of effectiveness of cutting edge techniques. Our findings reveal substantial disparities in the state of the art, with an exploit mitigation rate ranging from a low of 29% to a high of 74%. Our study identifies systemic limitations, such as inconsistent functionality preservation, that must be addressed in future research on program repair for smart contracts.

I. INTRODUCTION

THE cost of deploying a vulnerable smart contract on-chain can reach millions of dollars in financial losses. Hence, smart contract developers and auditors dedicate significant time and effort to ensuring that the contracts are free from vulnerabilities prior to deployment [34]. There are multiple ways smart contracts can be attacked. For example, a reentrancy problem is one of the ecosystem’s most notorious smart contract vulnerabilities and keeps being exploited [24].

A key reason why smart contract vulnerabilities persist is the lack of effective tooling. Developers frequently cite insufficient automated support as a major obstacle to writing secure contracts [2], [58]. One kind of tooling is Automated Program Repair (APR) for smart contracts [5], [35], which promises to automatically patch vulnerabilities, offering a scalable way to secure contracts and reduce reliance on expensive manual audits.

However, despite this promise, APR for smart contracts remains underdeveloped compared to general purpose program repair [54], [28]. Most notably, there is no accepted sound methodology for evaluating whether an automated repair smart

contract patch is effective [5], [35]. This methodological weakness has three critical consequences. First, researchers cannot reliably measure progress in the field because the published results are based on incompatible evaluation setups. Second, practitioners cannot confidently trust or adopt repair tools, as there is no solid evidence of their comparative effectiveness. Third, the community lacks consolidated knowledge, which impedes not just research but also training, education, and ecosystem-wide improvements.

Our paper fills this crucial methodological gap, as we present a framework for sound empirical comparison among APR tools. The framework is built over one of the most thorough literature reviews of the field, and a novel experimental methodology to validate patches and vulnerability mitigation through actual exploits. The core innovation is our focus on executable exploits as ground truth for patch validation. Rather than relying on static analysis or manual inspection, we verify repair effectiveness by attempting to execute exploits against the patched contracts. This approach provides concrete evidence of whether an automated fix to a vulnerable contract prevents the corresponding attack.

We reproduce 7 out of 20 state-of-the-art APR tools and apply our methodology in an extensive empirical study using a dataset of 143 vulnerable contracts. We manually craft 91 executable exploits for these contracts. Our evaluation reveals that effectiveness varies significantly from 29% to 74%, across different vulnerability types and repair strategies.

Our work sets the ground for further research on automated program repair for smart contracts by providing a rigorous theoretical and empirical comparison among APR tools. For researchers and practitioners alike, our evaluation framework and the publicly available exploit dataset provide an essential foundation for assessing and enhancing smart contract security.

To summarize, we make the following contributions:

- A comprehensive literature review of automated program repair for smart contracts.
- A new methodology for measuring repair effectiveness through performing actual smart contract exploit mitigation, based on real, executable exploits.
- A complete set of experiments for our methodology based on 143 contracts, 7 APR tools, and 91 exploits.
- A publicly available replication package: 1) set of exploits for the SmartBugs-Curated dataset, with a well-designed exploitation harness, sb-heists at <https://github.com/ASSERT-KTH/sb-heists>, 2) all generated patches and corresponding manual analysis results at <https://github.com/ASSERT-KTH/RepairComp/>.

¹The authors are ordered alphabetically and contributed as follows. All authors defined the research questions and designed the empirical study. M.J. led the execution of the empirical study, including designing and implementing the experimental framework, debugging and reproducing the repair tools, implementing the exploits, and conducting the experiments. S.B. contributed to the reproduction of tools, the implementation of the exploits, and the review of the related literature. S.B. and M.J. both contributed to interpreting, analyzing the experimental results and performing the manual analysis. M.M. supervised the project and provided methodological guidance. All authors participated in writing and revising the manuscript.

The rest of this paper is organized as follows. Section II presents the knowledge required to understand our study. Section III surveys the problem space of program repair for smart contracts. Section IV details our evaluation methodology. Section V presents our empirical results. Section VI discusses implications. Section VII presents the threats to validity of our work. Section VIII covers related work, and Section IX concludes.

II. BACKGROUND

This section introduces the key concepts needed to understand our experimental setup and the implications of our findings.

Smart Contracts. Smart Contracts are self-executing programs, deployed on blockchain networks, to enforce predefined rules without intermediaries. Contracts written in high-level smart contract languages like Solidity are compiled to bytecode for deployment and execution.

Solidity. Solidity is Ethereum's main smart contract programming language. While expressive and tailored for the EVM, misuse of constructs like fallback functions and low-level calls can lead to security vulnerabilities. Although newer versions have introduced stricter checks and safer defaults, certain vulnerabilities are tightly linked to the language's design and semantics.

Ethereum Virtual Machine. The Ethereum Virtual Machine (EVM) is a Turing-complete runtime environment that deterministically executes smart contract bytecode on the Ethereum blockchain. It operates as a transaction-based state machine, where all state transitions, including smart contract deployment and execution, are triggered by transactions.

Transactions. Transactions are initiated by an Externally Owned Account (EOA). An EOA, controlled by a user through a private key, can initiate transactions to transfer Ether, deploy smart contracts, or invoke functions on existing contracts. During execution, a called contract may interact with other contracts. These interactions, known as internal transactions, occur within the same atomic execution context where either all succeed or all fail.

Smart Contract Vulnerabilities. There is a wide range of smart contract vulnerabilities [22]. A notable example is reentrancy, which arises when a contract calls an external contract before updating its internal state. Attackers can exploit this by invoking recursive calls through fallback functions, enabling malicious behavior such as fund draining. The 2016 DAO hack, which resulted in \$50 million in stolen Ether, exemplifies the severity of such flaws. Other notable types of smart contract vulnerabilities include bad randomness [36], numerical defects [3], and price oracle manipulation [51].

III. THE PROBLEM SPACE OF PROGRAM REPAIR FOR SMART CONTRACTS

A. Overview

The goal of Automated Program Repair (APR) tools for smart contracts is to generate a patch for a smart contract that is affected by a vulnerability. These tools have 3 key components: 1) Detection: To repair a piece of code, the tool

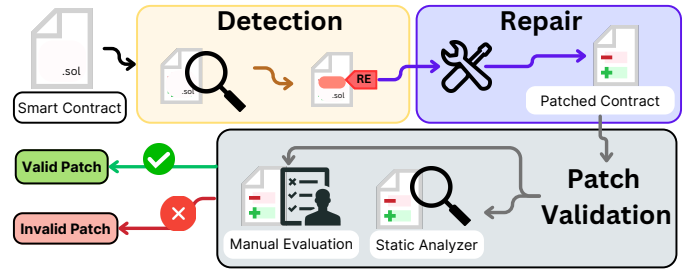


Fig. 1: Overview of the general workflow of an automated program repair tool for smart contracts.

must know what needs to be fixed, using some existing techniques. 2) Repair: Once the tool has received the localization of the vulnerability to be repaired, it must apply changes to the original code. 3) Patch Validation: An APR tool must output a patch that solves a vulnerability. There are different ways for assessing the validity of a patch.

Our literature review describes the tools from the research landscape. Table I consolidates our findings, highlighting the key components of each tool. Under “Repair-Level”, we specify the patch type that each tool generates (see subsection III-C). For “Detection”, “Repair Strategy”, and “Patch Validation” we follow the categories outlined in III-D, III-E and III-F, respectively.

B. Program Repair Tools for Smart Contracts

We now identify all notable program repair tools for smart contracts from the literature. Our systematic approach to select these tools is composed of 1) Searching for the keywords *program repair* and *smart contracts* on Google Scholar, 2) Reading all collected papers, 3) Performing citation analysis from their bibliographies, 4) Assessing whether they are compatible with the Ethereum stack for the sake of comparative evaluation, 5) Identifying tool names and links.

Through this rigorous investigation, we identified 19 potential APR tools for smart contracts. To the best of our knowledge, this is the most comprehensive list of smart contract APR tools at the time of writing, October 1st, 2024.

The first automated program repair tool was **SCRepair** [52], which proposes a mutation-based approach to generate patches and validate them using a test suite. Also working with mutation repair, **DeFinery** [46] is the first tool to incorporate formal verification for patch validation. Similarly, **SmartFix** [43] combines mutation with formal verification [42] and stands out by integrating a statistical model to prioritize likely correct patches.

A second family of approaches involves applying templates that are tailored for different vulnerabilities. **sGuard** [30] was the first contribution in this direction. **Aroc** [23] creates on-chain fixes with templates, we exclude it from the rest of this paper, due to its reliance on a modified EVM for repair. **HCC** [17], the Hardening Contract Compiler, applies templates to add security checks at compilation time, returning a source code patch. **ContractFix** [33] uses a multi-level template approach that operates at three levels of granularity: statement level, method level, and contract level. **TIPS** [4]

TABLE I: Overview of APR tools for smart contracts considered in this work. Each tool is described by its repair level, detection method (EXT: external, INT: internal, U-P: User-Provided), repair strategy, and patch validation approach (FV: Formal Verification, ME: Manual Evaluation, SA: Static Analysis). as reported in the original paper.

Tool	Level	Vulnerability Detection	Repair Strategy	Patch Validation
SCRepair [52] (2020)	source code	EXT (Slither [12], Oyente [27])	Mutation	SA
sGuard [30] (2021)		INT	Template	ME
Aroc [23] (2022)		U-P	Template	ME
DeFinery [46] (2022)		U-P	Mutation	FV
HCC [17] (2022)		INT	Template	SA&ME
ContractFix [33] (2023)		EXT (Slither [12], Securify [49], SmartCheck [45])	Template	SA&ME
GPT&BARD [32] (2023)		EXT (Slither [12], Oyente [27], Securify [49], HoneyBadger [48], Osiris [47], Mythril [7])	Generative	SA
SmartFix [43] (2023)		EXT (VeriSmart [42])	Mutation	FV&ME
SmartRep [57] (2023)		EXT	Supervised	N/A
SolGPT [29] (2023)		EXT (Slither [12])	Generative	SA
TIPS [4] (2023)		EXT or U-P (Slither [12], Mythril [7], SmartEmbed [15])	Template	SA&ME
ReenRepair [21] (2023)		INT	Template	ME&SA
ACFix [53] (2024)		INT	Generative	SA&ME
RLRep [19] (2024)		EXT (Slither [12], Oyente [27], Securify [49], Mythril [7])	Supervised	ME
sGuard+ [14] (2024)		INT	Template	ME
vFix [11] (2024)		EXT (Securify [49], Slither [12], SmartCheck [45])	Template	SA&ME
ContractTinker [50] (2024)		EXT (Slither [12])	Generative	SA&ME
SmartShield [55] (2020)	bytecode	EXT (Securify [49], Osiris [47], Mythril [7])	Template	SA
EVMPatch [39] (2021)		EXT (Oyente [27], Securify [49], Osiris [47], ECF [18], teEther [25], Maian [31], Sereum [38])	Template	SA
Elysium [13] (2022)		EXT (Oyente [27], Osiris [47], Mythril [7])	Template	SA

proposes a multi-template strategy where a vulnerability type can be fixed by multiple templates. **sGuard+** [14] takes the problem of accurately localizing the vulnerability by connecting bytecode, source code, and AST representation of the vulnerability in a contract. **vFix** [11] enhances the template applicability through pointer analysis and intra-procedural data-flow analysis. Finally, **ReenRepair** [21] focuses on reentrancy vulnerability and attempts to produce gas-optimized patches by reordering source code statements that correlate a READ and WRITE instruction in the bytecode.

Two APR tools for smart contracts use supervised learning: **SmartRep** [57] employs a double encoder to abstract the vulnerable method from the AST, and train a system from one-line fixes of GitHub commits. **RLRep** [19] applies a reinforcement learning approach with policy gradient, using a reward function that evaluates fixes based on compilation success, vulnerability detection, code entropy [37], and code similarity.

The latest trend in APR for smart contracts involves using Large Language Models (LLM). **GPT&BARD** [32] tests gpt-3.5 under three scenarios: vulnerable code with repair examples, solely vulnerable code, and vulnerable code with static analysis output. **SolGPT** [29] uses gpt-3.5-turbo, combining vulnerable code with static analysis results from Slither[12]. **ACFix** [53] addresses access control vulnerabilities by generating a role-pair taxonomy and using gpt-4 agents for generation and validation. **ContractTinker** [50] applies an agentic approach using gpt-4 to define a vul-

nerability mitigation plan and repair action.

Finally, there is an original line on binary APR tools, which repair EVM bytecode directly. **SmartShield** [55] extracts context from a control-flow-graph (CFG) and operates at the bytecode level to address missing checks on insecure operations and state changes after external calls. **EVMPatch** [39] apply repair on EVM bytecode with a trampoline technique, which is a technique to add new EVM instructions into empty code areas. Lastly, **Elysium** [13] applies context-aware patches using semantic-based techniques to gather relevant information like integer type and free storage space. All those binary tools are template-based approaches.

C. Detection & Repair Level

When studying program repair tools for smart contracts, the first classification to make is the level at which the tool operates. We identify two categories: source code level and bytecode level tools. Some tools can mix both in detection and/or repair.

The source code level refers to textual files in high-level programming languages (Solidity in this case, but could apply to other smart contract languages). The bytecode level refers to low-level instructions generated after compiling the smart contract source code. In this paper, it is specifically the EVM bytecode that is being considered.

Detection. When applying vulnerability detection on source code, the goal is to identify vulnerable lines of code. A

```

1 "name": "FibonacciBalance.sol",
2 "defect": [ {
3   "lines": [ 31, 38 ],
4   "category": "access_control" }]

```

Listing 1: User-Provided vulnerability detection information for TIPS [4]

potential downside of source code detection is the rate of false positives as the detectors fail to identify genuine problems in the code [1].

Vulnerability detectors at the bytecode level identify a particular instruction location in the EVM binary code. The benefit of detecting at the bytecode level is that it comes after compilation and optimization, so it reduces the false positive rate happening before compilation. A downside of detecting at this level is the lack of understandability of the binary code as it is not human-readable, and the difficulty mapping back to source code.

Repair. Repairing at the source code level is advantageous because it enables a developer to assess the quality of a patch with manual inspection. A downside is the difficulty of abstracting vulnerabilities in certain repair patterns as code can be written in thousands of different ways.

A common challenge of repairing at the bytecode level is the lack of high-level contextual information, including names and identifiers, lost during compilation.

Out of 19 selected tools, 16 repair at the source code level, and 3 tools apply repair at the bytecode level. We see that tools working on repair at the source code level are more popular. We believe this is due to the necessity of proposing patches that a human can assess and validate.

D. Vulnerability Detection

The first phase of an APR tool is localizing the vulnerability it aims to repair. Based on our literature review, we categorize vulnerability detection into 3 groups:

User-provided (U-P): The tool asks the user to categorize the contract vulnerabilities' according to a predefined format. For example, Listing 1 presents the format to follow for User-Provided detection on TIPS [4].

External (EXT): The tool relies on an external detector. For example, ContractFix [33] reuses Slither [12], Securify [49] and Smartcheck [45].

Internal (INT): The APR tool has its own mechanism for vulnerability detection. In this case, the tool contribution is both on the repair side and the detection side. An example of this category is sGuard [30] which proposes a new detection model and repair strategy.

We found that 5 tools use an internal detection mechanism, 3 rely on user-provided detection, and 12 use external detection. Also, source code tools that use external sources tend to rely on Slither [12] (7/10), and byte-code tools on Osiris [47] (3/3).

E. Repair Strategy

The main component of program repair tools is the one responsible for patch generation. The literature on repair for smart contracts relies on the following techniques.

Code Mutation: ([52], [46], [43]) This involves performing changes on the code according to specific transformation operators, in order to find a solution to a vulnerability. The scope for mutations can widely vary and therefore there is little overlap between the mutations considered in each paper.

Template: ([30], [23], [17], [33], [4], [55], [39], [13], [21], [14], [11]) This mechanism consists of creating templates designed for specific vulnerabilities. Tools can consider one template per vulnerability or many per family type. The second has been proven to improve repair capabilities [4].

Supervised Machine Learning: ([19], [57]) Predicts vulnerability patches based on previous fixes. A common challenge is the lack of such training data for smart contracts (labeled fixes) [19].

Generative Machine Learning (LLM): ([32], [29], [53], [50]) LLMs are trained on vast amounts of unsupervised data. It has been shown that they can be applied to smart contract coding tasks, including program repair. This approach involves feeding an LLM with source code and a vulnerability to get a patch. A challenge of this approach is the absence of logical explanation about the patch, with no logical reasoning about the process to come to the fix [53]

To sum up, the most popular technique in the literature is template-based, yet learning-based approaches have emerged in more recent studies.

F. Patch Validation

After generating patches, the next phase in APR is to validate them. Validation may involve ensuring functional correctness and/or vulnerability mitigation. This study focuses on vulnerability mitigation. In our survey of APR for smart contracts, we identified the following approaches for ensuring vulnerability mitigation:

Static Analysis (SA): The dominant technique in APR for smart contracts is running a static analyzer tool on the generated patches to statically check for vulnerability mitigation. SolGPT [29], for example, validates patches using Slither [12] to verify that the vulnerability has disappeared. The drawback of this technique is its reliance on the static analyzer's accuracy, as any false positive or false negative will affect the validation step of a tool.

Formal Verification (FV): Formal verification offers a deductive approach to analyzing the vulnerabilities in a program. It consists of mathematically proving the presence or absence of a vulnerability in a contract. In our study, DeFinery [46] and SmartFix [43] are the only tools that utilize this technique in the context of smart contract security.

Manual Evaluation (ME): Using their expertise, some authors manually determine if the vulnerability is still present in the patched code. An example of this validation technique is sGuard+ [14]. The drawbacks of manual evaluation are the lack of scalability and reproducibility.

Automated repair of vulnerabilities should work only with true positive vulnerabilities in a contract. True positive vulnerabilities mean that exists an exploit that can be executed to the contract [34]. To validate patches for vulnerability mitigation, one should use each corresponding exploit.

In RQ3 IV-D we propose an innovative validation technique by generating exploits for the vulnerable contracts in our study.

G. Recapitulation

To sum up, past contributions on APR for smart contracts exhibit considerable variations in their detection, repair, and validation methods. The 19 tools analyzed in our study rely on different detectors, target distinct vulnerability categories, and are evaluated on diverse, often incompatible benchmarks. It is impossible for the research and practitioner communities to truly understand where the state of the art is, and what the main strengths and weaknesses of each tool are. Our work addresses this gap by empirically testing APR tools on a common dataset and evaluating their ability to mitigate actual exploits in a sound manner.

IV. EXPERIMENTAL METHODOLOGY

A. Research Questions

We devise a novel experimental methodology to study APR for smart contracts. Our study is articulated around five research questions.

- RQ1 (scientific reproducibility): To what extent are research tools for APR smart contract available and executable?
- RQ2 (systematic comparison): How do APR tools for smart contracts perform on a common benchmark?
- RQ3 (exploit mitigation): To what extent do APR tools mitigate executable smart contract exploits?
- RQ4 (efficacy per vulnerability type): What is the mitigation effectiveness per vulnerability type?
- RQ5 (manual analysis): For patches that mitigate an exploit, to what extent can they be considered correct?

B. RQ1: Scientific Reproducibility

After consolidating the list of APR tools to assess (Table I), we proceed to systematically test all tools for basic executability and reproducibility. This experiment has the following steps:

- 1) **Availability:** For all papers, we search for the repository link in the article. When having an unsuccessful search, we reach out to the authors asking for the code.
- 2) **Installability:** We try to set up the environment, install the dependencies, and compile, if applicable, the tool's source code, per the provided documentation. When struggling with this step, we reach out to the authors asking for further instructions.
- 3) **Executability:** Finally, if the previous steps succeed, we test the tools by executing them using both the example inputs provided in their repositories and custom test contracts.

In cases where a step failed, our initial approach was to manually debug and resolve the errors, keeping the modifications as minimal as possible. Before deciding to drop a tool from further experiments, our last attempt is always to directly communicate with the authors.

This experiment provides an overview of reproducibility in smart contract APR, showing that results are reproducible within the tools' claimed operating conditions. It also identifies a set of executable APR tools that can be used for deeper investigation in subsequent research questions.

C. RQ2: Basic Repair Effectiveness

In this experiment, we aim to run all reproducible APR tools (see Table II) on a common benchmark. To our knowledge, this has never been done before. As a benchmark, we use the well-known SmartBugs-Curated dataset [8], which has been widely employed in related research. The dataset comprises 143 vulnerable smart contracts, each annotated with one of ten categories derived from the DASP taxonomy: reentrancy, access control, arithmetic, unchecked low-level calls, denial of services, bad randomness, front running, time manipulation, short addresses, and miscellaneous. SmartBugs-Curated is well appropriate for our study because its widespread use in previous research provides a foundation for comparison. Importantly, all the tools we evaluate are fully compatible with this dataset. This compatibility ensures a consistent evaluation framework and facilitates a fair comparison across tools, making SmartBugs-Curated well-suited for our experiment.

Listing 2 shows an example of a contract from the dataset with the reentrancy vulnerability in its `withdrawBalance()` function. On line 6, the contract sends funds via `call.value()` before zeroing the user's balance on line 9. This critical sequence allows an attacker to exploit the code by recursively calling the `withdrawBalance()` function through a fallback function, repeatedly withdrawing funds before the balance is updated, effectively draining more funds than their initial balance.

```

1 contract Reentrancy {
2     mapping (address => uint) userBalance;
3     ...
4     function withdrawBalance() {
5         // <yes> <report> REENTRANCY
6         if (!(msg.sender.call.value(userBalance[msg.sender])) {
7             throw;
8         }
9         userBalance[msg.sender] = 0;
10    }
11 }
```

Listing 2: Example of a SmartBugs-Curated smart contract with a reentrancy vulnerability.

On top of the existing benchmark infrastructure, we add a novel evaluation framework to assess program repair capability and patch validity. This patch assessment does not exist in SmartBugs-Curated. The patch assessment framework comprises five steps. Each step is described in detail below.

- 1) **Detection Accuracy:** We assess each tool's ability to detect the original vulnerability. SmartBugs-Curated uses the DASP taxonomy to classify vulnerabilities. For each

vulnerability category detected by the tools, we manually match it to the appropriate DASP class by reviewing the tool's description. Then, we compare the ground truth DASP class against the detected one on each contract.

- 2) **Patch Generation:** Each tool is configured according to its original instructions to generate patches. For each contract in the dataset, we generate and collect all patches produced by the tools.
- 3) **Patch Compilation:** We try to compile all the patches from the previous step using the appropriate compiler version. This step assesses the syntactic and typing correctness of the produced patches. Note that this step does not apply to binary APR tools.
- 4) **Differential Analysis:** We note that some tools sometimes do not change anything in the original contract. To rule out ineffective repairs, we compare each generated patch against the original contract file. This involves examining the diff files to ensure the patches effectively modify the original code.
- 5) **Consistency via Tool Detector:** Finally, we check whether the patch fixes the original vulnerability according to the tool's own detector. We compare each tool's vulnerability report of the original smart contract and its respective patch and check that the vulnerability is no longer detected. This means that the tool's detection and repair algorithms are consistent with each other.

D. RQ3: Exploit Mitigation Rate

Next, we evaluate the effectiveness of the considered tools in truly mitigating smart contract exploits.

As previously mentioned in subsection III-F Patch Validation, automated program repair for smart contracts aims at preventing exploits while maintaining functionality. Therefore, evaluating a patched contract's ability to mitigate an exploit is a strong criterion for repair effectiveness, stronger than, for example, the absence of statically detectable issues. To our knowledge, this concept of performing actual exploit mitigation has never been done at this scale.

There is a fundamental challenge to doing this: the absence of executable exploits. Existing vulnerability datasets, including SmartBugs-Curated, have vulnerabilities but no exploits. In this experiment, we first manually write successful exploits for our dataset and then run the exploits on the patched contracts to check whether the patch mitigates them.

Exploit Writing. We manually write executable exploits for SmartBugs-Curated dataset. Each exploit targets a specific vulnerability and is verified by violating a contract's core property: either safety properties (e.g., unauthorized token transfers) or liveness properties (e.g., denial of service). The manual creation of each exploit is time-blocked for 1 hour. We identify the attack vector by understanding the contract's logic, interface, and the labeled security weakness. We construct a precise transaction sequence that reliably exploits the vulnerability. When necessary, our exploits incorporate auxiliary smart contracts that are called attacker contracts, enabling the attack.

We propose an original methodology to assess exploit mitigation, illustrated in Figure 2. This methodology has two main steps: Functional Check and Exploit Check.

- 1) **Functional Check:** We note some patches repair vulnerabilities at the cost of breaking important contract functionality. For each exploit, we create multiple benign transactions that exercise the function without triggering the vulnerability. These are manually created by analyzing the contract's source code and associated metadata (such as historical transactions, if any). The benign transactions are designed to safely exercise the vulnerable code paths according to the contract's intended behavior, they do not test the full contract functionality. The correctness of these transactions is cross-checked between authors. These transactions must always succeed on a patched contract, confirming that the patch preserves basic functionality. If any benign transaction fails, the patch is considered invalid. To sum up, functional checks verify that the patched contract maintains its intended behavior for benign use cases.

This check is essential because the exploit harness only verifies attack success or mitigation, but does not detect if the patch breaks the contract. From a methodological standpoint, without functional checks, such broken patches can be misinterpreted as effective. Our work is the first to address this key methodological aspect in program repair for smart contracts.

- 2) **Exploit Check:** We use the Hardhat Framework [44] to ensure consistent execution and reproducibility of these exploits; each exploit is executed as a Hardhat test. The test orchestrates a structured sequence of transactions on a local blockchain. These transactions deploy the smart contracts and trigger their respective vulnerabilities. Finally, the test performs a verification step using pass/fail condition checks to confirm whether the exploit successfully compromised the contract. In other words, to check safety properties, assertions are added to detect invalid program states and violations of invariants. For liveness, the benign transactions ensure no deadlocks or interruptions of normal operations.

While exploit creation involves manual effort, it is a one-time cost. Once written, these exploits can be reused to automatically evaluate any number of patches from current and future APR tools, making the evaluation process itself highly scalable.

To answer RQ3, we focus on the following metrics: functional check failure rate and exploit mitigation rate. The failed functional check rate indicates the percentage of patches that break the contract's functionality. This helps identify patches that render the contract unusable for its designed purpose. The exploit mitigation rate measures the proportion of patches that successfully achieve two goals: they prevent the original vulnerability from being exploited while maintaining the contract's core functionality. This metric provides direct insight into how effectively each tool can generate patches that improve security without introducing functional regressions.

We note that none of the exploit generation tools, such as teEther [25], are capable of generating those exploits because they focus on very specific attacks that are not considered in the dataset used in this study.

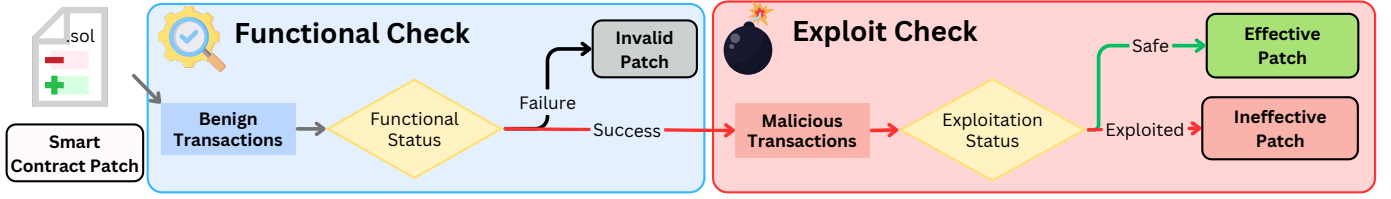


Fig. 2: RQ3: Our novel methodology for evaluation via actual, executable exploits, a core contribution to the field of smart contract repair.

E. RQ4: Efficacy per Vulnerability Type

Following, we aim to analyze the efficacy of vulnerability repair from the perspective of vulnerability types.

To address RQ4, we aggregate the results of RQ3 per vulnerability type, using the metadata provided in our dataset. This analysis examines exploit mitigation from two perspectives: the types of mitigated vulnerabilities and the performance of each tool with respect to the vulnerabilities it is explicitly designed to address.

This research question helps identify the vulnerability types effectively addressed by current tools and those that still require significant attention from future research.

F. RQ5: Manual Analysis of Patches

To deepen our understanding of patch quality, we conduct a manual evaluation of source code patches that mitigate an exploit in our dataset using two sampling strategies:

1) Tool-based Sampling: From the patches that successfully mitigate exploits per the automated check, we randomly select 10 samples produced by each source code level tool.

2) Vulnerability type based Sampling: To assess repair efficacy across vulnerability classes, we randomly selected 10 exploit-mitigating patches, if available, for each vulnerability type considered.

Each selected patch is manually examined to assess two key criteria: (1) whether it disrupts the expected normal behavior of the contract, and (2) whether it effectively mitigates the underlying vulnerability. The manual review is independently conducted by two authors, with disagreements resolved through discussion with a third author.

Additional factors considered during the evaluation include the correctness of the associated exploits and the potential introduction of new vulnerabilities.

V. EXPERIMENTAL RESULTS

A. RQ1: Scientific Reproducibility

Following the methodology outlined in Section IV-B, we present the findings from our reproducibility experiment. Out of an initial set of 19 tools, 15 tools have publicly available source code, while the source code for the remaining 4 tools was inaccessible. The reasons were copyright restrictions (2 cases) or the absence of a response from the authors (2 cases). Table II summarizes our efforts in installing and executing these 15 tools. We successfully installed 11 tools using their source code. We encountered issues such as missing or deprecated libraries with the remaining ones. This is the case

for SCRepair and ContractFix where required Python modules and NPM packages, respectively, were not available in the source code. DeFinery was not reproducible due to failure in installing its dependencies.

Among the 11 installed tools, 7 were successfully executed according to their provided usage examples and additional smart contracts, while 2 failed during execution due to runtime errors or crashes, and 2 could not be executed due to insufficient documentation and unclear input requirements. Notably all tools employing machine learning techniques failed the training execution. SmartRep execution fails during the training process, where it encounters an unexpected exception. RLRep fails in loading its necessary modules and the training process fails to update the model. vFix lacks sufficient documentation or examples on how to use detection reports to run the tool, making it infeasible to execute. ContractTinker successfully runs on the provided examples, yet we could not generalize its execution to other smart contracts due to missing documentation on required input formats, particularly regarding the expected structure of vulnerability reports.

Overall, we were able to execute 7/19 research tools from the literature. For them, we only had to make minor modifications to the code to address syntax errors or dependency issues.

Our results underscore challenges in research transparency (one-fourth of the tools presented in peer-reviewed articles lack publicly available code). Even when the code is available, there are serious reproducibility issues, as less than half were reproducible.

To improve this situation, we make the following recommendations. 1) Open-sourcing code with clear usage instructions, ideally on a platform like GitHub that supports version control and collaboration. We believe that GitHub facilitates error resolution compared to Anonymous GitHub or Zenodo. 2) Adhering to good coding practices, such as including a build file with dependencies and their versions. 3) For machine learning tools, provide the fully trained models alongside the training source code, as the training process is resource-intensive and prone to failure.

Answer to RQ1: To what extent are research tools for APR smart contract available and executable?

From an initial dataset of 19 APR tools, 15 have publicly available source code, 11 can be successfully installed, and 7 can be successfully executed end-to-end with only minor changes to the provided code.

TABLE II: Results for RQ1: Tools labeled according to their availability, and whether Installation and Execution was a success (✓) or failure (✗)

Tool	Availability	Installability	Executability	
SCRepair	GitHub	✗	-	
sGuard	GitHub	✓	✓	
DeFinery	GitHub	✗	-	
HCC	Not Found	-	-	
ContractFix	GitHub	✗	-	
GPT&BARD	Not Found	-	-	
SmartFix	Zenodo	✓	✓	
SmartRep	GitHub	✓	Training	Repair
			✗	✗
SolGPT	GitHub	✓	✓	
TIPS	GitHub	✓	✓	
ACFix	GitHub	✓	✗	
RLRep	GitHub	✓	Training	Repair
			✗	✗
sGuard+	Zenodo	✓	✓	
vFix	GitHub	✓	✗	
ContractTinker	GitHub	✓	✗	
SmartShield	Site	✓	✓	
EVMPatch	Non Available	-	-	
Elysium	GitHub	✓	✓	
ReenRepair	Non Available	-	-	

B. RQ2: Basic Repair Effectiveness

Table III shows the result of our large scale repair experiment on dataset SmartBugs-Curated per our evaluation methodology described in subsection IV-C. The columns correspond to the different steps of the methodology. The columns are Detected, Generated, Compilable, Different, and Consistent. In this table, column C stands for the number of vulnerable contracts with at least one patch that fulfills the criterion of the column, and P stands for the number of patches that satisfy the same criterion.

Detected C. Following our methodology we first check whether each tool detects the vulnerability according to the ground truth from the dataset. The accuracy varies considerably. SolGPT achieves the highest accuracy, by detecting 97/143 vulnerable contracts. In contrast, sGuard only identifies 35/143 contracts as vulnerable, the lowest accuracy among all tools.

On Table III we found for 119 contracts there is at least one tool correctly detecting the label vulnerability. It confirms that SolGPT has the best strategy for detecting the vulnerability. Also, we note that there is an overlap among detection. This suggests that an ensemble method for vulnerability detection may be valuable for wider coverage. Since no tool detects 24 contracts as vulnerable, the dataset SmartBugs-Curated is not yet exhausted for research.

Patch Generation. The second metric of the study focuses on patch generation. The dataset includes 143 contracts. TIPS achieves 140 patched contracts, followed closely by SolGPT

with 139. Notably, SolGPT and TIPS are the only tools able to generate multiple patches per contract. This leads to SolGPT producing 552 patches and TIPS generating 242 patches over the whole dataset, more than the total number of contracts. One contract (parity_wallet_bug_1.sol) did not obtain a single patch from any tool due to a solidity version required by the contract but incompatible with the experimental pipeline.

Compilable. For 142 out of 143 contracts, at least one patch compiles. sGuard+ and SmartFix achieve a 100% compilation success rate. The remaining three tools produce at least one non-compilable patch. The non-compilable patches in sGuard are due to a syntax error caused by an incorrect variable in the code. For SolGPT, some patches fail due to syntax errors stemming from non-code elements accidentally included in the LLM response, which is a known problem for LLM-based code generation. In TIPS, compilation fails for only two smart contracts, both due to syntax errors related to improper handling of string literals. Overall, the compilation results are good, all tools have a high ratio of compilable patches, showing reasonable engineering quality.

Different. Our fourth step in this experiment is to run a differential analysis. With this metric, we ensure that the produced patches actually modify the original code. sGuard+, SmartFix, SolGPT, and TIPS always produce patches different from the original contract, as expected. In contrast, sGuard achieved only 57% unique patches, with 46 contracts remaining identical to the original code. Similarly, Elysium produced 79% different patches, leaving 26 unchanged contracts, while SmartShield reached 100% uniqueness. Our results indicate the relevance of this metric, which is often overlooked in related research.

Consistent. Finally, we evaluate the consistency between the tool's detector and the patch generator. This means if the tool considers the vulnerability, initially detected in the contract, as disappeared for the generated patches. A patch is considered consistent if all instances of the detected vulnerability are undetected after patching. A contract is considered consistent if there is at least one patch that removes all instances of the detected vulnerability. Among the tools, SolGPT is consistent for 88/97 contracts, the highest absolute number across tools. TIPS followed closely with 81/82 consistently fixed contracts, while SmartFix achieved 48/51. sGuard+ gathered 70/70 consistent contracts achieving 100% consistency on the detected contracts. sGuard only achieves consistency for 2/35 contracts.

This discrepancy stems from differences in the tools' detectors. Specifically, sGuard's detector does not account for addition of protective code and continues to flag vulnerabilities based on low-level patterns, such as unsafe opcode sequences. Although sGuard and sGuard+ share similar template-based repair approaches, their detectors differ substantially, leading to distinct outcomes. These findings suggest that adopting more sophisticated and context-aware detectors can significantly improve consistency.

TABLE III: RQ2: Comparison of patches generated for each tool based on detection, compilation, modification, and vulnerability repair. **C** represents the total number of vulnerable contracts with at least one patch in that category, and **P** represents the number of patches in that category.

Level	Tool	detected C	generated		compilable		different		consistent	
			C	P	C	P	C	P	C	P
source code	sGuard	35	109	109	108 (99%)	108	62 (57%)	62	2/35 (6%)	2
	sGuard+	70	81	81	81 (100%)	81	81 (100%)	81	70/70 (100%)	70
	SmartFix	51	86	86	86 (100%)	86	86 (100%)	86	48/51 (94%)	48
	SolGPT	97	139	552	138 (99%)	527	139 (100%)	552	88/97 (91%)	332
	TIPS	82	140	242	138 (99%)	234	140 (100%)	242	81/82 (99%)	129
bytecode	Elysium	52	121	121	n/a	n/a	95 (79%)	95	52/52 (100%)	52
	SmartShield	59	135	135	n/a	n/a	135 (100%)	135	38/59 (64%)	38
Total Contracts		143	119	142	142		142		115	

Answer to RQ2: How do APR tools for smart contracts perform on a common benchmark?

Our evaluation shows that 142 contracts have multiple patches from diverse APR tools. Yet, we observe inconsistencies in detection accuracy and patches which severely undermine soundness. These findings highlight the urgent need for better methodologies, incl. robust patch validation, and reliable end-to-end repair pipelines.

C. RQ3: Exploit Mitigation Rate

We successfully manually create 91 exploits for the SmartBugs-Curated dataset. Each exploit targets one vulnerable contract in the dataset. For every exploit, we also write a robust, handwritten functional check to evaluate the functionality preservation of the patched contract. Table IV summarizes the results of our exploit experiments conducted on 91 contracts with known vulnerabilities.

Failing Functional Check Rate. Table IV presents the functional failure rates for each tool. The results reveal that certain patches disrupt the core functionality of the contracts.

Our findings show that 6 out of 7 tools fail to preserve functionality in some contracts, emphasizing the critical importance of functional checks when assessing smart contract patches. Our results reveal a clear dichotomy between bytecode-level and source-code-level repair tools. Bytecode tools (Elysium, SmartShield) exhibit significantly higher functional failure rates (26/91 and 21/91, respectively) compared to source-code tools (average 3/91). This suggests that operating at the bytecode level makes it substantially harder to preserve the original contract logic and functionality. While bytecode repair offers the advantage of working without source code, our findings indicate this comes at the cost of frequently breaking core contract functionality.

In contrast, template-based tools such as sGuard, sGuard+, and TIPS exhibit high functionality preservation, with low failure rates. This suggests that their template-based repair strategies are more effective in maintaining contract functionality. However, these templates are not universally applicable, and some patches still lead to functional breakdowns.

```

1 mapping (address => uint) userBalance;
2 ...
3 function withdrawBalance() {
4 +   userBalance[msg.sender] = 0;
5   if (!(msg.sender.call.value(userBalance[msg.sender])) {
6     throw;
7   }
8 -   userBalance[msg.sender] = 0;
9 }

```

Listing 3: Reentrancy vulnerability patched by TIPS [4].

```

1 mapping (address => uint) userBalance;
2 ...
3 function withdrawBalance() {
4 +   uint256 tmp_1 = userBalance[msg.sender];
5 +   userBalance[msg.sender] = 0;
6   if (!(msg.sender.call.value(tmp_1)) {
7     throw;
8   }
9 -   userBalance[msg.sender] = 0;
10 }

```

Listing 4: Reentrancy vulnerability patched by SmartFix [43].

For instance, Listings 3 and 4 both address reentrancy exploits but yield differing outcomes. Listing 3 illustrates a TIPS patch that mitigates reentrancy by setting the user's balance to zero before transferring it. While this approach prevents the exploit, it also renders the `withdrawBalance` function unusable, as it always transfers zero. Conversely, Listing 4 showcases a SmartFix patch for the same vulnerability. This patch stores the user's initial balance, updates it, and then transfers the stored value, successfully preventing reentrancy while preserving functionality. This example highlights the importance of functional checks in detecting patches that disrupt contract functionality, which the exploit harness alone cannot identify. Without functional checks, such broken patches could be misclassified as effective.

Exploit Mitigation Rate. Our exploits are attempted on every patched contract, with the corresponding results shown in Table IV. The effectiveness in mitigating exploits varies significantly. SolGPT achieves the highest mitigation rate with 67 patches (74% of sb-heists) mitigating the exploit, while SmartShield has the lowest with 26 exploits mitigated, representing 29% of the exploit dataset. There is a clear disparity in the tools' ability to effectively address smart con-

TABLE IV: RQ3: Number of patched contracts that failed functional checks and the total and unique exploits mitigated by each tool out of 91 exploits.

Tool	Failed Functional Checks	Mitigated Exploits	
		Total	Unique
sGuard	1 (1%)	30 (33%)	0
sGuard+	0 (0%)	44 (48%)	0
SmartFix	4 (4%)	48 (53%)	1
SolGPT	7 (8%)	67 (74%)	4
TIPS	1 (1%)	45 (49%)	4
Elysium	26 (29%)	29 (32%)	1
SmartShield	21 (23%)	26 (29%)	0

tract vulnerabilities. We note that no single tool successfully mitigates all exploits, there are still 11 contracts with an exploit that no tool succeeds in mitigating.

The third column of Table IV, “Unique” shows the exploits that only one tool can handle uniquely. SolGPT and TIPS mitigate the highest number of unique exploits (4), followed by SmartFix and Elysium, each with one exploit not covered by others. This analysis highlights the necessity for diverse mitigation strategies. We believe that industrial tools for smart contract repair will follow an ensemble based repair approach to maximize effectiveness.

Our results demonstrate that no single repair tool is universally optimal. While SolGPT achieves the highest mitigation rate (74%), template-based tools like TIPS best preserve functionality (only 1 failed functional check), and bytecode-level tools (e.g., SmartShield) offer the advantage of source-free repair but at the cost of higher failure rates (28%). This suggests that practitioners should not rely on a single tool but instead adopt ensemble strategies. For instance, they can prioritize template-based patches if they are available, and only use LLM-generated fixes for cases not covered by templates. Future work should 1) explore hybrid techniques to address the 11 still-unmitigated exploits and 2) improve bytecode-level repair, as functional correctness remains a critical challenge.

Answer to RQ3: To what extent do APR tools mitigate executable smart contract exploits?

This is the first ever experiment to demonstrate that APR tools for smart contracts actually mitigate smart contract exploits. SolGPT stands out with the highest mitigation rate, stopping 74% of the exploits. Next comes SmartFix, a mutation-based tool, which mitigates 53% of the exploits. Template-based tools such as TIPS (49%), sGuard+ (48%), and sGuard (33%) show moderate mitigation success but are notably better at preserving smart contract functionality. Our publicly available dataset of functional checks and exploits will help future researchers in conducting a strong, sound evaluation of APR for smart contracts.

D. RQ4: Efficacy per Vulnerability Type

The results of our analysis of mitigation effectiveness by vulnerability type are presented in Table V. For each vulnerability type in the dataset, the table reports the number of exploits in our sb-heists dataset, the number of mitigations per tool, as well as the aggregated mitigation counts and overall relative effectiveness. On each tool’s column, “n/a” stands for not applicable when the tool’s paper or documentation does not declare that it addresses that particular vulnerability type.

Unchecked Low-Level Calls. This is the most prevalent vulnerability type in the dataset, with 52 contracts and 20 exploits. Of the 7 reproducible tools in our experiment, 5 explicitly target this vulnerability. Our results show that 19/20 contracts have a patch that mitigates the unchecked low-level call vulnerability. Overall, this means that 95% of such vulnerabilities are properly mitigated, meaning that this problem can be considered solved.

Reentrancy. In SmartBugs-Curated, 31 contracts belong to the reentrancy family of vulnerability, 26 of them have a corresponding executable exploit in sb-heists. All 7 tools evaluated in this study are designed to address this vulnerability. Our result indicates that for all reentrant contracts, there is at least one patch that mitigates the exploit, which is arguably a strong result. This is an encouraging result for smart contract developers, who can use already detection and repair tools to handle reentrancy.

Access Control The access control vulnerability appears in 18 contracts in SmartBugs-Curated, with 16 associated exploits. In this study, 6 out of the 7 tools are designed to target this vulnerability type. Our results show that for 13 out of 16 exploits, there is at least one patch that successfully mitigates the issue, corresponding to a repair effectiveness of 81%. While this indicates meaningful progress, it also highlights the need for further research to fully address access control vulnerabilities.

Arithmetic This vulnerability category includes 15 contracts in the dataset, with 13 corresponding exploits. In this study, 6 out of the 7 reproducible tools are designed to address this category. Arithmetic vulnerability patching has a perfect 100% effectiveness. In addition, we note that this has also been fully handled at the compiler level, in release 0.8 of the Solidity compiler.

For the rest of the vulnerabilities in the dataset, we have a reduced number of contracts and corresponding exploits. For **Bad Randomness**, there are 4 exploits and a 100% effectiveness for exploit mitigation, **Time Manipulation** got 67% mitigation effectiveness over the 3 exploits, and **Front Running** one single mitigated exploit (33%). On the other hand, for **Denial of Service**, no tool could produce patches for this vulnerability. This calls for further research on these vulnerability types.

As shown in Table V, APR tools work well on Unchecked Low Level Calls, Reentrancy, Access Control, and Arithmetic vulnerabilities, with all those vulnerability types having an effectiveness higher than 80%. None of the tools explicitly targets Denial of Service or Front Running.

However, this does not necessarily imply zero patch generation, as demonstrated by Elysium and SmartShield, which

TABLE V: RQ4: Mitigation effectiveness per vulnerability type, tool and contract in dataset SmartBugs-Curated

Vulnerability Type	# Exploits	Tools							Aggregation	
		sGuard	sGuard+	SmartFix	SolGPT	TIPS	Elysium	SmartShield	# Mitigation	% Effectiveness
Unchecked Low-Level Calls	20	n/a	5	n/a	15	18	8	9	19	95%
Reentrancy	26	25	25	26	25	25	0	4	26	100%
Access Control	16	3	4	10	11	2	8	n/a (2)	13	81%
Arithmetic	13	2	10	12	9	n/a	7	9	13	100%
Bad Randomness	4	n/a	n/a	n/a	3	n/a	n/a (3)	n/a	4	100%
Denial of Service	4	n/a	n/a	n/a	n/a	n/a	n/a	n/a	0	0%
Time Manipulation	3	n/a	n/a	n/a	2	n/a	n/a (2)	n/a (1)	2	67%
Front Running	3	n/a	n/a	n/a	n/a	n/a	n/a (1)	n/a (1)	1	33%
Miscellaneous	2	n/a	n/a	n/a	2	n/a	n/a	n/a	2	100%
Total	91	30	44	48	67	45	29	26	80	88%

produced patches for Front Running. Since these tools provide bytecode patches, further analysis is out of scope for this study.

Overall, 88% of exploits are mitigated at least once, showing that research has made significant progress towards automatic mitigation of smart contract exploits. Yet, we bear in mind that the contracts in SmartBugs-Curated do not reflect the full complexity of real-world contracts.

Answer to RQ4: What is the mitigation effectiveness per vulnerability type?

Our study of exploit mitigation rate per vulnerability type demonstrates high effectiveness for most of them. Reentrancy, Arithmetic, and Bad Randomness exploits can all be mitigated. Unchecked Low-Level Calls and Access Control exploits are also mitigated with high effectiveness rates of 81% and 95%, respectively. These positive results show that the research community has made good progress, but also hint that SmartBugs-Curated may start to be an exhausted benchmark. Our results clearly motivate the construction of new benchmarks with 1) more vulnerability types (e.g. price manipulation vulnerabilities and 2) more real-world smart contracts.

E. RQ5: Manual Analysis of Patches

To assess the quality of the generated patches, we manually analyze a total of 111 patches. These include ten randomly selected patches per tool that produce source code fixes, as well as ten (when available) per vulnerability type.

All 111 patches successfully block the corresponding exploit, providing strong evidence for the construct validity of our automated validation framework. However, three patches generated by SolGPT [29] break legitimate functionality, revealing the challenge of designing comprehensive functional checks.

```

1 contract Proxy {
2   address owner;
3   ...
4   // <yes> <report> ACCESS_CONTROL
5   function forward(address callee, bytes _data) public {
6     - require(callee.delegatecall(_data));
7     + require(callee.call(_data));
8   }

```

Listing 5: Example of an incorrect patch for an access control vulnerability.

The first incorrect patch is illustrated in Listing 5. The patch targets an access control vulnerability in a proxy contract. Proxy contracts act as intermediaries by forwarding calls via `delegatecall`, allowing the code of another contract to execute in the proxy's storage context. If not properly restricted, `delegatecall` introduces critical risks: unauthorized callers may manipulate the storage of the proxy, potentially resulting in loss of funds or corrupted state. In the vulnerable contract, `delegatecall` is used without access control.

A correct patch requires checking if the address of the caller belongs to a set of authorized addresses before proceeding with the `delegatecall`.

The incorrect patch replaces the `delegatecall` with a `call`, likely under the assumption that it would prevent the vulnerability. However, this fundamentally alters the semantics of the contract. Unlike `delegatecall`, a `call` executes the code in the callee's storage context, meaning the patch eliminates the proxy functionality rather than fixing the vulnerability.

This case goes undetected by our framework because the exploit check confirms that no funds are lost through malicious use of the patched `forward` function, and the functional check verifies that the contract owner can successfully call this function. Yet, neither check is exhaustive enough to detect the loss of proxy behavior caused by the patch.

```

1 contract PoCGame {
2   address private whale;
3   uint256 totalDonated;
4   ...
5   function loseWager(uint256 am) internal {
6     // <yes> <report> UNCHECKED_LL_CALLS
7     whale.call.value(am)(bytes4(keccak256("donate()")));
8     + whale.transfer(am);
9     totalDonated += am;
10  }

```

Listing 6: Example of an incorrect patch uncaught by incomplete functional check.

The remaining two incorrect patches target contracts with similar unchecked low-level call vulnerabilities, which are generalized in Listing 6. In the contracts, the function `donate()` is invoked on an external contract `whale` using a low-level call. If this call fails, the contract does not handle the failure case, resulting in an incorrect update to the internal state variable `totalDonated`.

```

1 // <yes> <report> UNCHECKED_LL_CALLS
2 -whale.call.value(am)(bytes4(keccak256("donate()")));
3 +if(!whale.call.value(am)(bytes4(keccak256("donate()"))))
4 + { throw; }

```

Listing 7: Example of a correct patch that maintains functionality and mitigates the exploit.

Listing 7 shows a correct patch by TIPS [4] targeting the same contract, which retains the low-level call but adds a proper return value check.

The incorrectly proposed patch in Listing 5 replaces the low-level call with a direct `transfer` of Ether, which automatically reverts the transaction on failure, thereby preventing the state inconsistency. However, manual inspection on the `whale` contract's code reveals that `whale.donate()` includes additional logic beyond just receiving funds. Thus, this patch leads to an unintended behavior and results in an inconsistent state in the called contract `whale`.

This issue goes undetected by our functional checks, as the exploit check confirms that the vulnerability no longer leads to inconsistent state updates, and the functional check verifies that the game logic remains operational. However, neither check captures the loss of intended behavior due to the omitted logic in `whale.donate()`.

These cases illustrate that functional checks alone do not fully capture behavior preservation.

An interesting case is that of “Bad randomness”, where all reviewed patches mitigate the exploits by changing the hash function used for randomness. However, the new computation still follows the same insecure pattern [36], meaning that while the concrete exploit is blocked and the functional checks pass, the general problem remains. This highlights a key limitation of exploit-driven validation: although patches may successfully block concrete exploits, they may overlook the broader vulnerability class, particularly in cases where the exploitability depends on subtle execution characteristics.

We note that repair tools, especially learning-based ones, may introduce new vulnerabilities while fixing the target one. We explicitly check for such cases in our manual analysis of 111 patches. None of them introduces a new vulnerability.

Answer to RQ5: For patches that mitigate an exploit, to what extent can they be considered correct?

Out of 111 manually evaluated patches, 108 (97%) are found to be logically correct by three expert reviewers, demonstrating the reliability of our automated validation framework. Only three patches are deemed incorrect, with each failure being explained by corner-case limitations in our functional check mechanism; calling for future work on exhaustive functional checks.

VI. DISCUSSION

Our empirical evaluation of smart contract APR tools reveals critical insights about their capabilities, limitations, and directions for future research. We organize our discussion around three key themes that emerge from our empirical results.

A. Impact of Detection Accuracy on Repair Performance

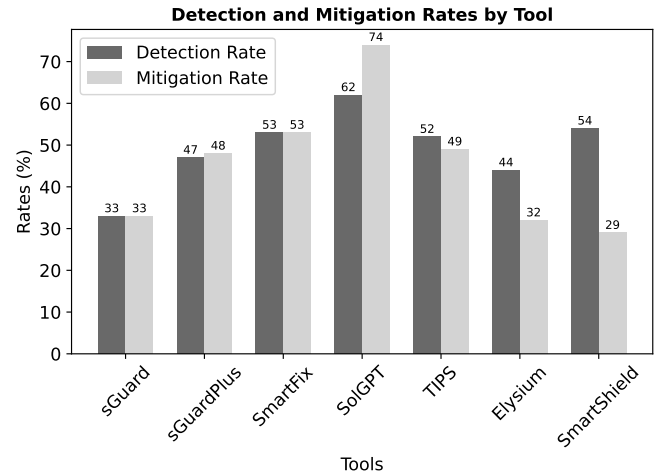


Fig. 3: Detection Rate and Exploit Mitigation Rate for each APR tool. Detection Rate represents the percentage of vulnerabilities with exploits (91) identified by each tool. Mitigation Rate reflects the percentage of exploits successfully mitigated out of the same total. Results are specific to vulnerabilities with exploits and, therefore, differ from Table III.

Smart contract vulnerability detection remains a significant challenge in the field. Previous studies have highlighted limitations in detection capabilities [1], [41]. These detection limitations directly affect automated fixes: one cannot repair what one cannot detect.

Our empirical results quantify this dependency between detection accuracy and repair success. Figure 3 presents a comparison of each tool's detection rate (the percentage of exploitable vulnerabilities detected out of 91 total) and mitigation rate (the percentage of exploits mitigated out of 91). For sGuard, SmartFix, and TIPS, the detection rate is very close to the mitigation rate, meaning high consistency. For Elysium

TABLE VI: Distribution of Unexploitable SmartBugs-Curated Contracts by Challenge Type.

Challenge Type	Number of Contracts
Theoretical Problem	30
Missing Exploit Entry Point	9
Solidity Version	8
Exceeded Analysis Time Limit	3
Honeytrap and Hash Cracking	2

and SmartShield, the detection rate is higher, suggesting that repair is harder at the binary code level.

Both sGuard+ and SolGPT surprisingly mitigate more than what they detect. sGuard+ mitigates one more exploit than the number of accurately detected vulnerabilities. Manual analysis reveals this happens because the tool identifies and fixes a vulnerability of a different type than the labeled one. The tool inadvertently mitigates the exploit targeting the labeled vulnerability by addressing this additional issue. SolGPT exhibits a higher disparity between detection and mitigation rates: the mitigation rate (74%) substantially exceeds the detection rate (62%). This is because the LLM changes the code beyond the given Slither warnings given in the prompt. In this case, the LLM leverages its training on large code repositories to regularize problematic code patterns directly, even when they are not explicitly prompted to do so. We note that this behavior could result from training data leakage, yet we are not aware of systematic repositories with SmartBugs-Curated patches. These results call for more research on LLM-based vulnerability detection for smart contracts.

B. Exploitability

A key insight of our study is that vulnerable does not necessarily imply exploitable, a conclusion consistent with prior work [34]. In our analysis of the 143 contracts from the SmartBugs-Curated dataset, we find that 91/143 (63.6%) can be successfully exploited using manually crafted exploits. The remaining 52 contracts (36.4%) could not be exploited, and the reasons for this are outlined in Table VI. The majority of unexploitable cases (30 contracts) involve theoretical vulnerabilities. These issues are either gated behind owner-only access or lead to no adverse effect. In practice, these contracts are safe. For 9 contracts, we found that the exploit entry point is missing. Although the vulnerability exists in theory, the contract lacks publicly accessible functions or interfaces that would allow an attacker to trigger the flaw. In 8 contracts, Solidity version inconsistencies posed challenges. 1 contract uses an outdated version (v0.4.11), unsupported by modern tooling (e.g., Hardhat), making practical testing infeasible. The other 7 were mislabeled with incorrect compiler versions, leading to semantic differences that nullified the originally reported vulnerabilities. 3 contracts exceeded our analysis time budget of one hour, preventing us from confirming exploitability within reasonable bounds. Finally, 2 contracts fall under unique cases. One is a honeytrap, a deceptive contract designed to lure and trap would-be attackers. The other requires hash

cracking as part of the exploit, which is computationally infeasible and beyond the scope of our analysis.

To sum up, we are the first to create an exploitable version of SmartBugs-Curated. From this process, the main takeaway is that it is hard to create a dataset of exploitable vulnerabilities. To our knowledge, there are very few such datasets, Code4rena [6] is one notable example. Curating exploitable vulnerability datasets is of high value for the research community.

C. Bytecode vs. Source-level Repair

Our results highlight a clear trade-off between bytecode-level and source-level APR tools. Bytecode-level tools like Elysium and SmartShield show the highest functional failure rates (29% and 23%, respectively), reflecting their struggle to preserve contract behavior, a critical concern for practitioner usage.

This limitation stems from bytecode's lack of semantic context. Without high-level constructs like variable names or control flow, these tools often apply low-level fixes that inadvertently alter intended behaviors. Moreover, patches generated at the bytecode level are difficult to interpret and to validate manually, further complicating correctness assurance.

Conversely, source-level tools such as SolGPT, SmartFix, and TIPS leverage semantic information to produce more accurate, human-readable patches. SolGPT, for instance, achieved the highest exploit mitigation rate (73%) alongside strong functionality retention, demonstrating the advantages of semantically informed repair.

However, source-level tools require access to the contract's source code, which is not always available in real-world deployments. In such scenarios, bytecode-level tools may be necessary due to their broader applicability, despite challenges in precision and validation.

D. Practical Recommendations for Future Research

Our findings yield several practical insights that can inform future work:

1. Relevance of Vulnerability Detection. Although vulnerability detection is not the primary goal of automated program repair tools, it plays a critical role in their success. We recommend the adoption of a standardized input format for vulnerabilities to enable interoperability between detection and repair components. For example, the format that TIPS uses to specify detected vulnerabilities (see Listing 1) is a good start towards a generic, machine readable format for specifying the vulnerabilities to be repaired. This would allow users to plug in newer detection tools without modifying the repair pipeline.

2. Patch Generation Strategies. Our study finds that the most effective approaches are generative AI (SolGPT) and mutation-based techniques (SmartFix), closely followed by template-based methods. We note that generative AI is a flexible generation engine. The model can be guided to emulate different repair paradigms, for example, few-shot prompting can emulate template-based repairs, while mutation-based strategies can be driven by prompting structured edits. We also foresee that future research can enhance generative AI

repair with information from complementary components such as symbolic execution and fuzzing.

3. Joint Patch Validation. In automated program repair for smart contracts, patch validation must account for both correctness and security. Proof-of-concept exploits offer a practical and effective way to assess whether a patch successfully mitigates a vulnerability. This type of validation provides strong security guarantees. However, functional correctness is also essential and robust repair tools should ensure both security and correctness. sb-heists provides APR research with a strong framework for validating patches for the sb-curated dataset. It is also a blueprint for future joint validation approaches for other benchmarks.

VII. THREATS TO VALIDITY

A. Internal Validity

Our findings depend on the reliability of tool execution and the exploit validation checks. In reproducing some APR tools, we encounter and resolve small errors. While done for enabling greater success, these fixes may affect the tool's original functionality, potentially influencing the detection and patching processes. Additionally, our manual vulnerability analysis and exploit creation are subject to human judgment, which may lead to inconsistencies in determining whether a vulnerability is exploitable. The functionality checks in our exploit evaluation may be incomplete, this could lead to misjudging the ability to mitigate exploits. To mitigate this risk, all code and results have been discussed between the authors.

B. External Validity

The composition of the dataset directly influences the findings. The SmartBugs-Curated dataset includes both real and artificial contracts, which may introduce biases since artificial contracts might not fully represent the complexity or nuances of real-world contracts.

From an empirical perspective, the execution environment is also a threat for generalization. The execution environment, using a local blockchain, may differ from the behavior of contracts on a live blockchain, potentially affecting the exploitability and repair effectiveness measurements. To mitigate this threat, we use the industrial grade hardhat framework, used by the majority of the smart contract industry.

C. Representativeness of Considered Tools

The APR tools we included in this comparative study represent 35% (7 out of 20) of the academic tools we found. This is because we did not manage to find or execute all existing tools, due to availability and scientific reproducibility issues.

We believe that those 7 tools we consider are representative of the state of the art for the following reasons. First, they were published across 3 out of the 4 years of active research in this area, hence capturing the evolution of the field. Second, they span fundamentally different repair strategies incl. template-based, mutation, and generative AI approaches.

VIII. RELATED WORK

In this section, we discuss related work in the areas of Smart Contract Repair Surveys and Smart Contract Vulnerabilities.

A. Surveys

There are notable surveys on smart contract repair. Chu et al. [5] categorize repair tools into two types: off-chain tools, which address vulnerabilities before deployment, and on-chain tools, which repair deployed contracts. While the survey highlights the advantages of each approach, it does not provide any discussion or experiment of actual vulnerability mitigation. Qian et al. [35] compare repair methods and the types of vulnerabilities each tool targets. Other recent surveys have studied vulnerability repair and mitigation [20], [26], yet none of them has performed an empirical, quantitative comparison as we have done in this paper.

Our work is the first to empirically evaluate the effectiveness of these APR tools at generating patches (subsection IV-C), the first to validate functionality preservation and actual mitigation of vulnerabilities (subsection IV-D).

B. Smart Contract Vulnerabilities

An early study on vulnerabilities in smart contracts was conducted by Chen et al. [2], who identified 20 types of vulnerabilities. Later, the work of Zou, et al. [58] surveyed 232 practitioners involved in smart contract development. Among the many challenges raised, they highlighted the lack of proper tooling for ensuring security in smart contracts. These calls for action led to the development of various detection tools, which have since been the subject of numerous studies [1], [16], [24].

Ghaleb et al. [16] systematically evaluate smart contracts analysis tools by injecting known vulnerabilities. Chaliasos et al. [1] examine the effectiveness of vulnerability detection tools in terms of their potential to prevent attacks. Khan et al. [24] propose a comprehensive literature review and classification of detection tools, which, to our knowledge, represents the most recent contribution in the rapidly evolving field of vulnerability detection. From an empirical perspective the work of Durieux et al. [10], Di Angelo et al. [9], and Sendner et al. [41] have assessed vulnerability detection tools by testing them on previously labeled datasets. These studies collectively emphasize the need for more accurate detection methods, as current tools tend to generate a high number of false positives and a concerning number of false negatives.

To understand the high rate of false positives on vulnerability detection, Perez et al. [34] propose to study the correlation between exploited and vulnerable contracts. They analyze on-chain smart contracts labeled as vulnerable by various detection tools and their actual exploitation status. The authors conclude that many contracts labeled as vulnerable are not exploitable. Similarly, Sayeed et al. [40] analyze attack vectors for seven vulnerabilities and highlight the inefficiency of widely used detection tools in preventing real-world exploits. Jiao et al. [22] provide hints of the attack vectors for eleven known vulnerabilities and categorize the different approaches for detection with their corresponding challenges.

Finally, Zhang et al. [56] demonstrate the value of detecting vulnerabilities from an exploit perspective, discovering 15 zero-day exploitable vulnerabilities in a study of 516 smart contracts.

Our work contributes to the much less researched area of smart contract repair. We are the first to demonstrate effectiveness through exploit mitigation, offering a sound evaluation of patch effectiveness.

IX. CONCLUSION

This paper has introduced a novel framework for studying automated program repair of smart contracts using executable exploits. By creating exploits and designing sound functional checks, we provide the first concrete proof of vulnerability mitigation in the domain of smart contract security, offering the first reliable assessment of the state of the art of Automated Program Repair for smart contracts. Our methodology and reusable artifacts provide solid foundations for future research on automated repair for smart contracts.

ACKNOWLEDGMENT

This work was partially supported by the WASP Program funded by Knut and Alice Wallenberg Foundation, and by the Swedish Foundation for Strategic Research (SSF). Some computation was enabled by resources provided by the National Academic Infrastructure for Supercomputing in Sweden (NAISS).

REFERENCES

- [1] Stefanos Chaliasos, Marcos Antonios Charalambous, Liyi Zhou, Rafaila Galanopoulou, Arthur Gervais, Dimitris Mitropoulos, and Benjamin Livshits. Smart contract and defi security tools: Do they meet the needs of practitioners? In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, ICSE '24*, New York, NY, USA, 2024. Association for Computing Machinery.
- [2] J. Chen, X. Xia, D. Lo, J. Grundy, X. Luo, and T. Chen. Defining smart contract defects on ethereum. *IEEE Transactions on Software Engineering*, 48(01):327–345, jan 2022.
- [3] Jiachi Chen, Zhenzhe Shao, Shuo Yang, Yiming Shen, Yanlin Wang, Ting Chen, Zhenyu Shan, and Zibin Zheng. Numscout: Unveiling numerical defects in smart contracts using llm-pruning symbolic execution. *IEEE Transactions on Software Engineering*, pages 1–16, 2025.
- [4] Qianguo Chen, Teng Zhou, Kui Liu, Li Li, Chunpeng Ge, Zhe Liu, Jacques Klein, and Tegawendé F. Bissyandé. Tips: towards automating patch suggestion for vulnerable smart contracts. *Automated Software Engineering*, 30(2):31, Sep 2023.
- [5] Hanting Chu, Pengcheng Zhang, Hai Dong, Yan Xiao, Shunhui Ji, and Wenrui Li. A survey on smart contract vulnerabilities: Data sources, detection and repair. *Information and Software Technology*, 159:107221, 2023.
- [6] Code4rena — code4rena.com. <https://code4rena.com/>.
- [7] ConSenSys. The mythrill tool. <https://github.com/ConsensSys/mythrill>.
- [8] Monika di Angelo, Thomas Durieux, João F. Ferreira, and Gernot Salzer. SmartBugs 2.0: An execution framework for weakness detection in Ethereum smart contracts. In *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE 2023)*, 2023. to appear.
- [9] Monika di Angelo, Thomas Durieux, João F. Ferreira, and Gernot Salzer. Evolution of automated weakness detection in ethereum bytecode: a comprehensive study. *Empirical Software Engineering*, 29(2):41, 2024.
- [10] Thomas Durieux, João F. Ferreira, Rui Abreu, and Pedro Cruz. Empirical review of automated analysis tools on 47,587 ethereum smart contracts. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering, ICSE '20*, page 530–541, New York, NY, USA, 2020. Association for Computing Machinery.
- [11] Pengcheng Fang, Peng Gao, Yun Peng, Qingzhao Zhang, Tao Xie, Dawn Song, Prateek Mittal, Sanjeev Kulkarni, Zhuotao Liu, and Xusheng Xiao. Vfix: Facilitating software maintenance of smart contracts via automatically fixing vulnerabilities. In *2024 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 13–24. IEEE, 2024.
- [12] Josselin Feist, Gustavo Grieco, and Alex Groce. Slither: a static analysis framework for smart contracts. In *2019 IEEE/ACM 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB)*, pages 8–15. IEEE, 2019.
- [13] Christof Ferreira Torres, Hugo Jonker, and Radu State. Elysium: Context-aware bytecode-level patching to automatically heal vulnerable smart contracts. In *Proceedings of the 25th International Symposium on Research in Attacks, Intrusions and Defenses, RAID '22*, page 115–128, New York, NY, USA, 2022. Association for Computing Machinery.
- [14] Cuifeng Gao, Wenzhang Yang, Jiaming Ye, Yinxing Xue, and Jun Sun. sguard+: Machine learning guided rule-based automated vulnerability repair on smart contracts. *ACM Trans. Softw. Eng. Methodol.*, 33(5), jun 2024.
- [15] Zhipeng Gao, Lingxiao Jiang, Xin Xia, David Lo, and John Grundy. Checking smart contracts with structural code embedding. *IEEE Transactions on Software Engineering*, 47(12):2874–2891, 2021.
- [16] Asem Ghaleb and Karthik Pattabiraman. How effective are smart contract analysis tools? evaluating smart contract static analysis tools using bug injection. In *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2020*, page 415–427, New York, NY, USA, 2020. Association for Computing Machinery.
- [17] Jens-Rene Giesen, Sebastien Andreina, Michael Rodler, Ghassan O. Karame, and Lucas Davi. Practical mitigation of smart contract bugs. *arXiv preprint arXiv:2203.00364*, 2022.
- [18] Shelly Grossman, Ittai Abraham, Guy Golan-Gueta, Yan Michalevsky, Noam Rinetzy, Mooly Sagiv, and Yoni Zohar. Online detection of effectively callback free objects with applications to smart contracts. *Proc. ACM Program. Lang.*, 2(POPL), dec 2017.
- [19] Hanyang Guo, Yingye Chen, Xiangping Chen, Yuan Huang, and Zibin Zheng. Smart contract code repair recommendation based on reinforcement learning and multi-metric optimization. *ACM Trans. Softw. Eng. Methodol.*, 33(4), apr 2024.
- [20] Wejdene Haouari, Abdelhakim Senhaji Hafid, and Marios Fokaefs. Vulnerabilities of smart contracts and mitigation schemes: A comprehensive survey. *arXiv preprint arXiv:2403.19805*, 2024.
- [21] Ruiyao Huang, Qingni Shen, Yuchen Wang, Yiqi Wu, Zhonghai Wu, Xiapu Luo, and Anbang Ruan. Reenrepair: Automatic and semantic equivalent repair of reentrancy in smart contracts. *Journal of Systems and Software*, 216:112107, 2024.
- [22] Tengyun Jiao, Zhiyu Xu, Minfeng Qi, Sheng Wen, Yang Xiang, and Gary Nan. A survey of ethereum smart contract security: Attacks and detection. *Distrib. Ledger Technol.*, 3(3), September 2024.
- [23] Hai Jin, Zeli Wang, Ming Wen, Weiqi Dai, Yu Zhu, and Deqing Zou. Aroc: An automatic repair framework for on-chain smart contracts. *IEEE Transactions on Software Engineering*, 48(11):4611–4629, 2022.
- [24] Zulfiqar Ali Khan and Akbar Siami Namin. A survey of vulnerability detection techniques by smart contract tools. *IEEE Access*, 12:70870–70910, 2024.
- [25] Johannes Krupp and Christian Rossow. Teether: gnawing at ethereum to automatically exploit smart contracts. In *Proceedings of the 27th USENIX Conference on Security Symposium, SEC'18*, page 1317–1333, USA, 2018. USENIX Association.
- [26] Nayantara K Kumar, Niranjana V Honnunar, M Sharwari Prakash, and J J Lohith. Vulnerabilities in smart contracts: A detailed survey of detection and mitigation methodologies. In *2024 International Conference on Emerging Technologies in Computer Science for Interdisciplinary Applications (ICETCS)*, pages 1–7, 2024.
- [27] Loi Luu, Duc-Hiep Chu, Hrishi Olickel, Prateek Saxena, and Aquinas Hobor. Making smart contracts smarter. In *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security*, pages 254–269, 2016.
- [28] Martin Monperrus. The living review on automated program repair. Technical Report hal-01956501, HAL Archives Ouvertes, 2018.
- [29] Emanuele Antonio Napoli and Valentina Gatteschi. Evaluating chatgpt for smart contracts vulnerability correction. In *2023 IEEE 47th Annual Computers, Software, and Applications Conference (COMPSAC)*, pages 1828–1833, 2023.
- [30] Tai D. Nguyen, Long H. Pham, and Jun Sun. Sguard: Towards fixing vulnerable smart contracts automatically. In *2021 IEEE Symposium on Security and Privacy (SP)*, pages 1215–1229, 2021.

- [31] Ivica Nikolić, Aashish Kolluri, Ilya Sergey, Prateek Saxena, and Aquinas Hobor. Finding the greedy, prodigal, and suicidal contracts at scale. In *Proceedings of the 34th Annual Computer Security Applications Conference, ACSAC '18*, page 653–663, New York, NY, USA, 2018. Association for Computing Machinery.
- [32] Marco Ortu, Giacomo Ibbia, Claudio Conversano, Roberto Tonelli, and Giuseppe Destefanis. Identifying and fixing vulnerable patterns in ethereum smart contracts: A comparative study of fine-tuning and prompt engineering using large language models. Available at SSRN: <https://dx.doi.org/10.2139/ssrn.4530467>, 2023.
- [33] Pengcheng Peng, Yun, Qingzhao, Tao, Dawn, Prateek, Sanjeev, Zhuotao, and Xusheng. Contractfix: A framework for automatically fixing vulnerabilities in smart contracts. *arXiv preprint arXiv:2307.08912*, 2023.
- [34] Daniel Perez and Benjamin Livshits. Smart contract vulnerabilities: Vulnerable does not imply exploited. In *30th USENIX Security Symposium (USENIX Security 21)*, pages 1325–1341, 2021.
- [35] Peng Qian, Rui Cao, Zhengguang Liu, Wenqing Li, Ming Li, Lun Zhang, Yufeng Xu, Jianhai Chen, and Qinning He. Empirical review of smart contract and defi security: Vulnerability detection and automated repair. *arXiv preprint arXiv:2309.02391*, 2023.
- [36] Peng Qian, Jianting He, Lingling Lu, Siwei Wu, Zhipeng Lu, Lei Wu, Yajin Zhou, and Qinning He. Demystifying random number in ethereum smart contract: Taxonomy, vulnerability identification, and attack detection. *IEEE Trans. Softw. Eng.*, 49(7):3793–3810, July 2023.
- [37] Baishakhi Ray, Vincent Hellendoorn, Saheel Godhane, Zhaopeng Tu, Alberto Bacchelli, and Premkumar Devanbu. On the “naturalness” of buggy code. In *Proceedings of the 38th International Conference on Software Engineering, ICSE '16*, page 428–439, New York, NY, USA, 2016. Association for Computing Machinery.
- [38] Michael Rodler, Wenting Li, Ghassan Karame, and Lucas Davi. Sereum: Protecting existing smart contracts against re-entrancy attacks. In *Proceedings of the Network and Distributed System Security Symposium (NDSS'19)*, 2019.
- [39] Michael Rodler, Wenting Li, Ghassan O. Karame, and Lucas Davi. Evmpatch: Timely and automated patching of ethereum smart contracts. In *USENIX Security Symposium*, 2020.
- [40] Sarwar Sayeed, Hector Marco-Gisbert, and Tom Caira. Smart contract: Attacks and protections. *IEEE Access*, 8:24416–24427, 2020.
- [41] C. Sendner, L. Petzi, J. Stang, and A. Dmitrienko. Large-scale study of vulnerability scanners for ethereum smart contracts. In *2024 IEEE Symposium on Security and Privacy (SP)*, pages 220–220, Los Alamitos, CA, USA, may 2024. IEEE Computer Society.
- [42] Sunbeom So, Myungho Lee, Jisu Park, Heejo Lee, and Hakjoo Oh. Verismart: A highly precise safety verifier for ethereum smart contracts. *2020 IEEE Symposium on Security and Privacy (SP)*, pages 1678–1694, 2019.
- [43] Sunbeom So and Hakjoo Oh. Smartfix: Fixing vulnerable smart contracts by accelerating generate-and-verify repair using statistical models. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2023*, page 185–197, New York, NY, USA, 2023. Association for Computing Machinery.
- [44] The hardhat framework. <https://hardhat.org/>, 2024. Accessed: 2024-11-25.
- [45] Sergei Tikhomirov, Ekaterina Voskresenskaya, Ivan Ivanitskiy, Ramil Takhaviev, Evgeny Marchenko, and Yaroslav Alexandrov. Smartcheck: Static analysis of ethereum smart contracts. In *2018 IEEE/ACM 1st International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB)*, pages 9–16, 2018.
- [46] Palina Tolmach, Yi Li, and Shang-Wei Lin. Property-based automated repair of defi protocols. In *Proceedings of ASE '22*, 2023.
- [47] Christof Ferreira Torres, Julian Schütte, and Radu State. Osiris: Hunting for integer bugs in ethereum smart contracts. In *Proceedings of the 34th Annual Computer Security Applications Conference, ACSAC '18*, page 664–676, New York, NY, USA, 2018. Association for Computing Machinery.
- [48] Christof Ferreira Torres, Mathis Steichen, and Radu State. The art of the scam: demystifying honeypots in ethereum smart contracts. In *Proceedings of the 28th USENIX Conference on Security Symposium, SEC'19*, page 1591–1607, USA, 2019. USENIX Association.
- [49] Petar Tsankov, Andrei Dan, Dana Drachler-Cohen, Arthur Gervais, Florian Bünzli, and Martin Vechev. Securify: Practical security analysis of smart contracts. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, CCS '18*, page 67–82, New York, NY, USA, 2018. Association for Computing Machinery.
- [50] Che Wang, Jiashuo Zhang, Jianbo Gao, Libin Xia, Zhi Guan, and Zhong Chen. Contractinker: Llm-empowered vulnerability repair for real-world smart contracts. In *ACM International Conference on Automated Software Engineering (ASE 2024)*, 2024.
- [51] Rui Xi, Zehua Wang, and Karthik Pattabiraman. Pomabuster: Detecting price oracle manipulation attacks in decentralized finance. In *2024 IEEE Symposium on Security and Privacy (SP)*, pages 3923–3942, 2024.
- [52] Xiao Liang Yu, Omar Al-Bataineh, David Lo, and Abhik Roychoudhury. Smart contract repair. *ACM Trans. Softw. Eng. Methodol.*, 29(4), sep 2020.
- [53] Lyuye Zhang, Kaixuan Li, Kairan Sun, Daoyuan Wu, Ye Liu, Haoye Tian, and Yang Liu. Acfix: Guiding llms with mined common rbac practices for context-aware repair of access control vulnerabilities in smart contracts. *arXiv preprint arXiv:2403.06838*, 2024.
- [54] Quanjun Zhang, Chunrong Fang, Yuxiang Ma, Weisong Sun, and Zhenyu Chen. A survey of learning-based automated program repair. *ACM Trans. Softw. Eng. Methodol.*, 33(2), December 2023.
- [55] Yuyao Zhang, Siqi Ma, Juanru Li, Kailai Li, Surya Nepal, and Dawu Gu. Smartshield: Automatic smart contract protection made easy. In *2020 IEEE 27th International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pages 23–34, 2020.
- [56] Zhuo Zhang, Brian Zhang, Wen Xu, and Zhiqiang Lin. Demystifying exploitable bugs in smart contracts. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, pages 615–627, 2023.
- [57] Xiacong Zhou, Yingye Chen, Hanyang Guo, Xiangping Chen, and Yuan Huang. Security code recommendations for smart contract. In *2023 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pages 190–200, 2023.
- [58] Weiqin Zou, David Lo, Pavneet Singh Kochhar, Xuan-Bach Dinh Le, Xin Xia, Yang Feng, Zhenyu Chen, and Baowen Xu. Smart contract development: Challenges and opportunities. *IEEE Transactions on Software Engineering*, 47(10):2084–2106, Oct 2021.